UNIVERSIDADE DO ESTADO DE MATO GROSSO – UNEMAT · FACULDADE DE CIÊNCIAS EXATAS E TECNOLÓGICAS

Avaliação 2 – Aplicação CRUD com Página de Consulta

FACET-SNP-310 – Frameworks Modernos para Desenvolvimento de Sistemas · Turma 01 · 2026.2

Prof. Ivan Luiz Pedroso Pires

VALOR	DIVULGAÇÃO	PRAZO DE ENTREGA	FORMAÇÃO
10,0 pontos	**30/09/2026**	**14/10/2026, 23h59**	**Individual ou até 4**

1. Objetivo

Aplicar os conteúdos da Unidade 02 – **Vue 3 + Vite, Vuetify, Vue Router e Axios** – no desenvolvimento de uma aplicação web completa. Tomando como base o projeto construído em aula (tela *Gerenciar Clientes* com `v-data-table`, `v-dialog` e validação com `vee-validate`, menu lateral com `v-navigation-drawer` e rotas), cada equipe deve desenvolver um **CRUD** e, além dele, uma **página de consulta** que exiba todos os dados cadastrados e permita filtrá-los.

2. Tema e entidade

- **Tema livre** (ex.: biblioteca, estoque, pet shop, eventos, frota, academia, clínica etc.). Não será aceito o exemplo de *Clientes* da aula apenas copiado ou com os mesmos campos.
- A entidade principal deve ter **no mínimo 6 campos** além do `id`, contemplando pelo menos estes tipos: **texto**, **número**, **data**, **seleção** (lista de opções – `v-select` / `v-autocomplete`) e **booleano** (`v-switch` / `v-checkbox`).
- A variedade de tipos é obrigatória porque é ela que permite construir filtros diferentes na página de consulta.

3. Requisitos obrigatórios

3.1 CRUD (página de gerenciamento)

- Listagem dos registros em tabela (`v-data-table`) com ações de editar e excluir por linha.
- Cadastro e edição em formulário (ex.: `v-dialog`), com **validação** dos campos (obrigatoriedade, formato, valores mínimos/máximos) e mensagens de erro em português.
- Exclusão com **confirmação** antes de remover o registro.
- Retorno visual ao usuário após salvar/excluir (ex.: `v-snackbar`).

3.2 Página de consulta com filtros (requisito central desta avaliação)

- Página própria, com rota própria, que exibe **todos os registros e todos os campos** da entidade (em tabela, cards ou lista). É uma página de leitura: não precisa editar nem excluir.
- **Busca textual geral** (um campo que procura o termo em vários campos ao mesmo tempo).
- **Pelo menos 3 filtros por campo específico, de tipos diferentes**, por exemplo: seleção (categoria/situação), intervalo numérico (preço de/até) ou intervalo de datas (de/até) e booleano (ativo/inativo).
- Os filtros devem funcionar **combinados** (todos aplicados ao mesmo tempo) e deve haver um botão **Limpar filtros**.
- Exibir a **quantidade de resultados** encontrados e uma mensagem amigável quando nenhum registro atender aos filtros.

3.3 Compartilhamento e persistência dos dados

- A página de consulta deve exibir os **mesmos dados** manipulados pelo CRUD: o que for cadastrado, editado ou excluído em uma página precisa aparecer na outra.

- Os dados devem **persistir após recarregar** a página, usando `localStorage` (`JSON.stringify` ao salvar e `JSON.parse` ao carregar).
- Recomenda-se concentrar a leitura e a gravação em um único arquivo (ex.: `src/services/storage.ts`) usado pelas duas páginas. **Pinia não é exigido nesta avaliação.**

3.4 Consumo de API com Axios

- Utilizar **Axios** para consumir ao menos uma API pública integrada ao formulário do CRUD. Exemplos vistos em aula: **ViaCEP** (preencher endereço a partir do CEP) ou **IBGE** (carregar estados e municípios em `v-select`).
- Indicar carregamento (`:loading`) e tratar erros (CEP inválido, falha de rede).

3.5 Navegação, componentes e interface

- Rotas com **Vue Router** para, no mínimo: página inicial, CRUD e consulta, acessíveis por um menu.
- Componentização: separar partes reutilizáveis (ex.: formulário, barra de filtros, menu) em componentes com `props` / `emits`.
- Interface construída com componentes do **Vuetify**, organizada e utilizável em telas menores.
- Projeto em Vue 3 + Vite, executando com `npm install` e `npm run dev`.

4. Critérios de avaliação e distribuição da nota

3,0	**3,0**	**1,0**	**1,0**	**1,0**	**1,0**
CRUD	Consulta e filtros	Dados compartilhados	Axios / API	Rotas, componentes e Vuetify	Repositório e README

#	Critério	O que será verificado	Pontos
1	**CRUD**		**3,0**
1.1	Listagem	Tabela com todos os registros, cabeçalhos legíveis e ações por linha.	0,5
1.2	Cadastro	Formulário com todos os campos, usando o componente adequado a cada tipo.	0,5
1.3	Edição	Formulário carrega os dados do registro e salva as alterações corretamente.	0,5
1.4	Exclusão	Remove o registro correto, com confirmação prévia.	0,5
1.5	Validação	Campos obrigatórios e formatos validados; mensagens claras; não salva dados inválidos.	0,5
1.6	Retorno ao usuário	Mensagem de sucesso/erro após salvar, editar e excluir (ex.: `v-snackbar`).	0,5
2	**Página de consulta com filtros**		**3,0**
2.1	Exibição completa	Rota própria; mostra todos os registros com todos os campos, bem formatados (datas, valores, booleanos).	0,5
2.2	Busca textual	Um campo de busca que pesquisa em mais de um campo, sem diferenciar maiúsculas/minúsculas.	0,5
2.3	Filtros por campo	No mínimo 3 filtros de tipos diferentes (seleção, intervalo numérico/data, booleano) funcionando corretamente.	1,0
2.4	Combinação e limpeza	Filtros aplicados em conjunto (reativos, via `computed`) e botão para limpar todos.	0,5
2.5	Retorno ao usuário	Contador de resultados e mensagem quando nenhum registro é encontrado.	0,5

#	Critério	O que será verificado	Pontos
3	**Compartilhamento e persistência dos dados**		**1,0**
3.1	Dados compartilhados	CRUD e consulta usam a mesma fonte de dados; alterações em uma página aparecem na outra.	0,5
3.2	Persistência	Dados gravados no `localStorage` e mantidos após recarregar a página.	0,5
4	**Consumo de API com Axios**		**1,0**
4.1	Integração	Requisição com Axios a uma API pública integrada ao formulário (ViaCEP, IBGE ou outra).	0,5
4.2	Carregamento e erros	Indicador de carregamento e tratamento de erro (`try/catch`) com aviso ao usuário.	0,5
5	**Rotas, componentização e interface**		**1,0**
5.1	Vue Router e menu	Rotas configuradas e navegação funcional entre início, CRUD e consulta.	0,5
5.2	Componentes e Vuetify	Componentes reutilizáveis com props/emits; interface Vuetify organizada e responsiva.	0,5
6	**Repositório e documentação**		**1,0**
6.1	README	Membros (ou nome do aluno), tema, funcionalidades e instruções para executar o projeto.	0,5
6.2	Histórico de commits	Commits ao longo do período, com mensagens descritivas. Em grupo, todos os membros devem ter commits.	0,5
	Total		**10,0**

Cada item pode receber pontuação **integral** (atende completamente), **parcial** (atende com falhas ou de forma incompleta) ou **zero** (não implementado ou não funciona).

5. Entrega

1. O trabalho pode ser feito **individualmente ou em grupo de até 4 pessoas**. Os requisitos são os mesmos.
2. A entrega é o **link do repositório no GitHub**, que deve ser:
   - **público**; ou
   - **privado e compartilhado** com o usuário `ivanlppires` (*Settings* → *Collaborators* → *Add people*).
3. **Todos os membros do grupo devem enviar o link no SIGAA**, na tarefa da Avaliação 2.
4. Prazo: **14/10/2026 (quarta-feira), até 23h59**. Será avaliado o último commit feito até esse horário; commits posteriores serão desconsiderados.
5. Não versionar a pasta `node_modules` (mantenha o `.gitignore` do projeto).

Modelo mínimo do README.md

```
# Nome do Projeto

Avaliação 2 — FACET-SNP-310 — Frameworks
Modernos para Desenvolvimento de Sistemas
(2026.2)

## Integrantes
- Nome Completo 1 — @usuario-github
- Nome Completo 2 — @usuario-github
(se individual: apenas o nome do aluno)

## Tema

Breve descrição do sistema e da entidade
cadastrada.

## Funcionalidades
- CRUD de ...
- Página de consulta com filtros por ...
- API consumida: ...

## Como executar
npm install
npm run dev
```

Atenção – situações que zeram a nota:

- Aluno que **não enviar o link no SIGAA** ou cujo nome **não constar no README**.
- Repositório **inacessível** (privado e não compartilhado com `ivanlppires`) ou link quebrado.
- Trabalhos **copiados** entre grupos/alunos: nota zero para todos os envolvidos.

6. Checklist antes de enviar

☐ Entidade com 6 ou mais campos (texto, número, data, seleção e booleano).
☐ CRUD completo, com validação, confirmação de exclusão e snackbar.
☐ Página de consulta com busca geral, 3 filtros de tipos diferentes, limpar filtros e contador de resultados.
☐ CRUD e consulta compartilham os dados; dados persistem no localStorage ao recarregar.
☐ Axios consumindo API pública, com loading e tratamento de erro.
☐ Menu com rotas para início, CRUD e consulta.
☐ Projeto roda com `npm install` e `npm run dev` em um clone limpo.
☐ README com integrantes e instruções; repositório público ou compartilhado com `ivanlppires`.
☐ Link enviado no SIGAA por **todos** os integrantes.